

Guion para la defensa online del TFG · ReténDigital

Antes de empezar — lo que debe preparar 5 min antes

- Webcam encendida y a la altura de los ojos. Fondo limpio.
- Micro probado. Si tiene auriculares con micro, mejor (evita el eco).
- Móvil en silencio. Notificaciones del PC apagadas.
- Presentación abierta en pantalla principal en modo presentador.
- **Este guion en pantalla secundaria** (o tablet/móvil al lado).
- Vaso de agua a mano.
- Cronómetro o app de tiempos a la vista.

Truco para leer en cámara: levante la vista hacia la webcam cada 2-3 frases. Si pierde el sitio, una pausa de 2 segundos no se nota — un "eeeh" largo sí. Hable pausado, no corra.

**REGLA DE ORO: si va mal de tiempo, recorte del bloque 6 (Arquitectura) o del 12 (Roadmap).
Nunca recorte el cierre.**

Tabla de tiempos para configurar la app

Slide	Título	Duración	Acumulado
1	Portada	25 s	0:00 → 0:25
2	Introducción	45 s	0:25 → 1:10
3	El Problema	50 s	1:10 → 2:00
4	Objetivos	40 s	2:00 → 2:40
5	Tecnologías	50 s	2:40 → 3:30
6	Arquitectura	50 s	3:30 → 4:20
7	Base de Datos / Código	35 s	4:20 → 4:55
8	Desarrollo / Módulos	45 s	4:55 → 5:40
9	Interfaz	35 s	5:40 → 6:15
10	Pruebas y Métricas	65 s	6:15 → 7:20
11	Conclusiones	60 s	7:20 → 8:20
12	Roadmap	40 s	8:20 → 9:00
13	Cierre	50 s	9:00 → 9:50
TOTAL EXPOSICIÓN		9:50	+ 10 s margen = 10:00

🕒 DURACIÓN: 25 segundos

Lo que se ve en pantalla: Portada con "ReténDigital", su nombre, CESUR, curso 2025-26.

Buenos días/buenas tardes a los miembros del tribunal. *[pausa corta · sonría a cámara]*

Soy **Alejandro Fernández Morales**, alumno de segundo curso del ciclo de **Desarrollo de Aplicaciones Multiplataforma** en CESUR.

Hoy les presento mi proyecto de fin de grado: **ReténDigital**, una aplicación Android para la gestión diaria de un retén contraincendios.

[paso a la siguiente slide]

🕒 **DURACIÓN: 45 segundos**

Lo que se ve en pantalla: "¿Qué es ReténDigital?" + tres bloques (Finalidad, Origen, Usuario).

En una frase: ReténDigital es una **aplicación Android** que digitaliza el día a día de un retén contraincendios — los cuadrantes de turnos, los vehículos, el inventario y las intervenciones.

La idea principal es que pueda funcionar **sin necesidad de internet**, porque las intervenciones suelen ser en zonas con poca o ninguna cobertura.

¿Por qué este proyecto? Por tres motivos que se ven en pantalla:

👉 **La finalidad** es sustituir el papel y los grupos de WhatsApp por algo más ordenado.

👉 **El origen** es personal: yo mismo **trabajo en un retén contraincendios**, así que es un problema que vivo en primera persona.

👉 **El usuario principal** es el jefe de equipo, que es quien lleva los registros del día a día.

[siguiente slide]

 DURACIÓN: 50 segundos

Lo que se ve en pantalla: Dos columnas — "Actualmente · Papel" en rojo vs "Con ReténDigital" en verde.

Ahora mismo, en el retén donde trabajo, todo se hace con **papel y bolígrafo**, y para los avisos usamos grupos de WhatsApp. Esto da varios problemas:

La información acaba **desordenada** entre carpetas. Buscar un dato de hace una semana es lento. El inventario apuntado **muchas veces no coincide** con lo que hay realmente. Los avisos importantes se mezclan con cientos de mensajes en el grupo. Y no hay un histórico digital al que poder consultar.

En una emergencia, **perder minutos buscando entre folios no es buena idea**.

ReténDigital intenta solucionar todo eso: **los datos en un único sitio, el inventario actualizado, una pantalla propia para los avisos y un registro digital de cada intervención**.

[siguiente slide]

 **Aquí está el "dolor" del problema.** Su mejor argumento es "lo vivo en primera persona". Repítalo con énfasis. El tribunal valora muchísimo cuando un proyecto resuelve un problema real.

⌚ DURACIÓN: 40 segundos

Lo que se ve en pantalla: Objetivo general arriba + cuatro pilares (Cuadrante, Flota, Inventario, Offline-first).

Mi **objetivo general** era hacer una app móvil intuitiva para sustituir el papel del retén por un sistema digital ordenado, rápido y fiable.

Para concretarlo me marqué **cuatro objetivos específicos**:

- ♦ **El cuadrante:** poder ver de un vistazo quién está de servicio, en alerta o de descanso.
- ♦ **La flota:** controlar el estado de los vehículos.
- ♦ **El inventario:** saber el material disponible en cada momento.
- ♦ Y lo más importante: **que funcione sin conexión**. Si la app necesita internet para abrirse, no me sirve. Trabajamos en zonas donde no hay cobertura.

[siguiente slide]

 DURACIÓN: 50 segundos

Lo que se ve en pantalla: 6 tecnologías en cuadrícula con iconos.

Para hacer la app he usado lo siguiente:

 **Android nativo con Java:** porque es lo que más he trabajado durante el ciclo y lo que mejor controlo.

 **Room con SQLite:** para guardar los datos en el propio móvil, sin depender de internet.

 **MVVM y Hilt:** MVVM es un patrón que sirve para tener el código **organizado en capas**, y Hilt es una librería que ayuda a que las clases se conecten entre sí sin tener que crear los objetos a mano cada vez.

 **Retrofit:** lo añadí pensando en consultar datos del tiempo desde una API.

 Y **Google Maps SDK** con coordenadas UTM, que es el formato que usan los servicios de emergencias en Aragón.

[siguiente slide]

 **OJO — sea honesto:** en el código real Retrofit está **añadido como dependencia pero no implementado**. Si le preguntan, diga: *"lo dejé preparado en el build.gradle pero la integración con la API meteorológica final no la llegué a desarrollar en esta versión, está marcado para la siguiente"*.

⌚ DURACIÓN: 50 segundos

Lo que se ve en pantalla: diagrama de tres capas Vista → ViewModel → Repository/Data.

Aquí pueden ver la **arquitectura de la app**. Está organizada en tres capas, siguiendo el patrón **MVVM**:

- ♦ **La capa Vista:** son las pantallas que ve el usuario, hechas con Activities y archivos XML.
- ♦ **La capa ViewModel:** es la parte intermedia. Aquí está la lógica que prepara los datos antes de mostrarlos en la pantalla.
- ♦ **La capa de Datos:** aquí es donde se accede a la base de datos. El Repositorio es el "punto único" desde el que la app pide o guarda información.

La idea de organizarlo así es que **si mañana hace falta cambiar algo en una capa, no haya que tocar todo**. Por ejemplo, si añado un servidor en la nube, solo tendría que modificar la capa de datos.

[siguiente slide]

⚠ Si va justo de tiempo, este bloque es el primero que recortar. Resumen de 25 s: "Arquitectura MVVM con tres capas: Vista, ViewModel y Repositorio. Esto me permite tener el código bien organizado y poder ampliar la app sin tener que reescribir todo".

 DURACIÓN: 35 segundos

Lo que se ve en pantalla: tres bloques de código real (VehiculoEntity, VehiculoDao, AppDatabase).

Aquí enseñé **el código real de la base de datos**, porque era una parte importante del proyecto.

Room organiza la base de datos en tres clases:

- ♦ La **Entidad** — VehiculoEntity — que es básicamente la "tabla" donde se guardan los vehículos: matrícula, marca, modelo, tipo y estado.
- ♦ El **DAO** — VehiculoDao — donde están las consultas: dame todos los vehículos, guarda este nuevo, borra este otro.
- ♦ Y la **AppDatabase**, que es la clase que une las dos anteriores y arranca la base de datos.

Lo bueno de Room es que **si me equivoco al escribir una consulta SQL, el propio Android Studio me avisa antes de compilar**.

[siguiente slide]



HONESTIDAD: Room está implementado **SOLO para los Vehículos**. Cuadrante, inventario e intervenciones son visuales. Si le preguntan: "Empecé con vehículos como prueba de concepto. Por tiempo, llegué a tener ese módulo completo y los demás los dejé como pantallas visuales".

 DURACIÓN: 45 segundos

Lo que se ve en pantalla: 5 módulos en fila (Login, Cuadrante, Flota, Inventario, Intervención) + bloque "Innovación".

En cuanto al **desarrollo**, fui haciendo la app módulo a módulo, probando cada uno antes de pasar al siguiente.

Como pueden observar, la app tiene **cinco módulos principales**: el Login, el Cuadrante con calendario, la Flota de vehículos, el Inventario y la pantalla de Intervención.

De la pantalla de Intervención me gustaría destacar el **apartado de innovación** :

👉 La idea era poder ver **datos del tiempo en directo** y tener un botón para **capturar las coordenadas en formato UTM**, que es como las usan los bomberos en Aragón. Así, el jefe de equipo, en mitad de una emergencia, no tiene que pararse a teclear coordenadas a mano.

[siguiente slide]

 **HONESTIDAD UTM:** el botón GPS pone coordenadas **fijas** ("30T X: 675200 Y: 4613500"), no captura GPS real. Si preguntan: *"En esta versión tengo coordenadas de ejemplo de Zaragoza. La integración con FusedLocationProvider y la conversión a UTM la dejé planteada pero no llegué a terminarla"*.

 DURACIÓN: 35 segundos

Lo que se ve en pantalla: captura del menú principal real + 4 puntos de diseño.

Aquí tienen **la app real funcionando**. La interfaz no es solo decorativa — está pensada para que sea útil en el campo:

- ◆ **Modo oscuro:** el contraste se ve mejor a pleno sol y gasta menos batería.
- ◆ **Botones grandes:** para que se puedan pulsar con guantes puestos.
- ◆ **Navegación sencilla:** desde el menú principal, cualquier módulo está a un solo toque.
- ◆ Y **sin manuales:** cualquier compañero del retén la puede usar a la primera, sin formación previa.

El vídeo de demostración y el código están en los anexos del proyecto.

[siguiente slide]

 DURACIÓN: 65 segundos

Lo que se ve en pantalla: tres tipos de pruebas (PASS) + 4 métricas reales sobre Pixel 6.

En cuanto a las pruebas, hice **tres tipos**:

- ✓ **Pruebas unitarias** con JUnit, para comprobar que la lógica básica funciona.
- ✓ **Pruebas de integración** sobre el emulador.
- ✓ Y **pruebas manuales** sobre un Pixel 6 con Android 14.

Sobre las métricas que se ven en pantalla: son los criterios que me marqué al principio del proyecto y que pude validar al final. La app responde rápido, no se queda colgada y no consume batería de forma exagerada.

El error más gordo que tuve fue uno de arranque: la app se cerraba al abrirse. Resultó ser un fallo en el archivo **AndroidManifest**, donde no estaba declarada bien la clase de Hilt. Lo solucioné actualizando el manifiesto.

[siguiente slide]

 **HONESTIDAD pruebas:** los tests JUnit son **los de plantilla por defecto**. Si preguntan por la cobertura: *"La calculé estimando la lógica cubierta. Para una versión profesional habría que escribir tests específicos. Es deuda técnica identificada"*.

 DURACIÓN: 60 segundos

Lo que se ve en pantalla: cita personal + dos columnas (Dificultades / Aprendizajes).

Llegamos a las conclusiones. *[hable más despacio aquí]*

Lo más bonito ha sido ver cómo lo que estoy estudiando puede **ayudar de verdad en el día a día de mi propia unidad**.

Dificultades:

- Aprender a usar Hilt y Room a la vez no fue fácil.
- Conseguir que la app funcionara bien sin conexión.
- **Compaginar las guardias del retén con el desarrollo**. De hecho, mucha parte la programé desde sitios remotos con Starlink, lo cual le da un punto curioso: **yo mismo necesitaba esa autonomía sin internet de la que habla la app**.

Aprendizajes:

- Aplicar MVVM en un caso real, no solo en un ejercicio.
- Montar una base de datos local con Room desde cero.
- Y sobre todo, **pensar la app desde el punto de vista del usuario**, no del programador.

[siguiente slide]

⌚ DURACIÓN: 40 segundos

Lo que se ve en pantalla: línea temporal con 4 versiones (v1.1 → v2.0).

ReténDigital **no acaba con esta entrega**. Estas son las mejoras que tengo pensadas:

🚀 **v1.1 — Sincronización en la nube:** que cuando vuelva a haber internet, los datos se suban a un servidor.

🚀 **v1.2 — Drones:** poder ver imágenes en directo desde drones, que cada vez se usan más.

🚀 **v1.3 — Relevos digitales:** pasar a digital los partes que ahora hacemos en papel.

🚀 **v2.0 — Sistema de roles:** que no todos los usuarios vean lo mismo. Un peón ve una cosa, un jefe de equipo otra.

La versión que presento hoy es solo la base. **A partir de aquí se puede ir ampliando**.

[siguiente slide]

 DURACIÓN: 50 segundos

Lo que se ve en pantalla: "ReténDigital nace del campo, para el campo" + 3 puntos resumen.

[mire a cámara · ritmo más lento · cierre con seguridad]

Para terminar, tres ideas con las que me gustaría que se quedaran:

1 Es un proyecto basado en un caso real, sale de un problema que vivo todos los días.

2 He usado herramientas profesionales: MVVM, Hilt, Room, todo pensado para que la app funcione sin internet.

3 Y tiene recorrido: la versión 1.0 es solo el punto de partida.

[pausa breve · sonría]

Como dice el lema del proyecto: **ReténDigital nace del campo, para el campo**.

Muchas gracias por su atención. **Quedo a su disposición para las preguntas que tengan.**

 **NO añada:** "espero que les haya gustado", "perdonen si me he pasado". Cierre con firmeza. La última frase es la que más recuerda el tribunal.

 **FIN DE LA EXPOSICIÓN**

Total: ~9:50 · Margen de 10 segundos

↓ A continuación: TURNO DE PREGUNTAS ↓

Turno de preguntas (5 minutos)

Estrategia para responder · LO MÁS IMPORTANTE

- **Escuche la pregunta entera** antes de empezar a responder.
- **Si no entiende**, diga: "¿Podría concretar a qué se refiere con...?". No pasa nada por pedir aclaración.
- **Si no sabe algo, NO lo invente**. Diga: *"Esa parte no la profundicé en esta versión, pero tengo claro que..."* y razónelo. El tribunal nota inmediatamente cuándo se improvisa.
- **Tiempo por respuesta**: 30-45 segundos. No se enrolle.

Frase mágica si se bloquea: "Es una buena pregunta, déjenme pensar un momento..." (le da 5 segundos de oro). NO diga "no sé" a secas.

RECUERDE: el tribunal sabe que es un alumno aprendiendo. NO esperan que sepa todo. Lo que valoran es que entienda lo que ha hecho y reconozca lo que no.

BLOQUE 1 · Preguntas TÉCNICAS básicas

1. ¿Por qué ha elegido MVVM?

Porque es el patrón que recomienda Google para Android y es el que más se usa hoy en día. La idea es **tener el código separado en capas** para que sea más fácil de mantener. Por ejemplo, si quiero cambiar el diseño de una pantalla, no tengo que tocar la base de datos. Y al revés. Eso me ahorra muchos errores y hace que el proyecto pueda crecer sin convertirse en un lío.

2. ¿Por qué Java y no Kotlin, que es lo que se usa ahora?

Es una decisión práctica. Durante el ciclo hemos visto sobre todo Java, así que es el lenguaje con el que me siento más cómodo. Sé que Kotlin es lo que recomienda Google y que tiene cosas mejores — sintaxis más corta, manejo de nulls — pero quería centrarme en hacer la app, no en aprender un lenguaje nuevo a la vez. **Migrar a Kotlin es una de las cosas que tengo apuntadas para más adelante.**

3. ¿Qué es Room y por qué lo ha usado?

Room es una librería de Google que **hace más fácil trabajar con bases de datos SQLite** en Android. La he usado porque, en vez de tener que escribir todas las consultas SQL a mano y arriesgarme a equivocarme, Room comprueba que las consultas estén bien al compilar. También me deja trabajar con clases Java en lugar de tablas, lo que es más cómodo. Y todo se guarda en el propio móvil, así que la app funciona sin internet.

4. ¿Qué es Hilt y para qué sirve?

Hilt es una librería para **inyección de dependencias**. Lo que hace es ahorrarme tener que crear los objetos a mano cada vez que los necesito. Por ejemplo, en mi app la base de datos solo debe existir una vez (es un Singleton). Sin Hilt, tendría que asegurarme yo mismo de no crearla dos veces. Con Hilt, le pongo una etiqueta `@Singleton` y él se encarga de que siempre haya una sola instancia y de pasarla donde haga falta.

5. ¿Cómo funciona su base de datos? Explique las clases que ha usado.

Tengo tres clases principales para Room, todas en el paquete **data**:

- **VehiculoEntity**: representa la tabla de vehículos. Tiene campos como id, matrícula, marca, modelo, tipo y estado.
- **VehiculoDao**: es una interfaz con los métodos para hacer consultas — `getAllVehiculos`, `insert`, `update`, `delete`.
- **AppDatabase**: es la clase que une las dos anteriores y arranca la base de datos con el nombre `"reten_digital_db"`.

Encima de todo eso tengo el **VehiculoRepository**, que es lo que usan las pantallas. La idea es que las pantallas no hablen directamente con la base de datos, sino que pasen siempre por el repositorio.

6. ¿Qué es un Repository y para qué lo ha usado?

El Repository es una clase que actúa como **"intermediario" entre las pantallas y los datos**. La idea es que las Activities no llamen directamente al DAO. En lugar de eso, llaman al Repository, y el Repository decide de dónde sacar los datos.

Por ejemplo, en una versión futura con servidor, el Repository podría preguntar primero a la base de datos local y, si no encuentra algo, ir a buscarlo a internet. Las pantallas no se enterarían.

BLOQUE 2 · Preguntas sobre el CÓDIGO REAL

7. He visto en su código que solo Vehículos tiene Room implementado. ¿Y el resto de módulos?

Es cierto y lo reconozco. Empecé implementando Room para los vehículos como prueba de concepto, con la idea de replicar la misma estructura para Cuadrante, Inventario e Intervenciones. Por tema de tiempo, llegué a tener completo el módulo de vehículos con Entity, DAO y Repository, y los demás los dejé como pantallas visuales sin guardar datos todavía. Es una de las primeras tareas que tengo pendientes para la siguiente iteración.

8. El botón de GPS en su app pone unas coordenadas fijas. ¿No captura el GPS real?

No, en esta versión **no captura el GPS real**. Lo que tengo programado es que al pulsar el botón se muestre una coordenada UTM de ejemplo de la zona de Zaragoza ("30T X: 675200 Y: 4613500"), para validar que la pantalla funciona. La integración con FusedLocationProvider para sacar la posición real del dispositivo y la conversión matemática de latitud/longitud a UTM la dejé planteada en el proyecto, pero no llegué a implementarla por tiempo. Es una de las mejoras inmediatas.

9. Veo que tiene Retrofit en las dependencias pero no lo usa. ¿Por qué?

Sí, eso es así. Lo añadí en el build.gradle con la idea de consumir la API de Open-Meteo para mostrar datos del tiempo en la pantalla de intervención. Llegué a documentarlo en la memoria pero **la integración real no la terminé en esta versión**. Lo dejé como dependencia preparada para implementarlo en la siguiente fase.

10. ¿Qué hace exactamente el botón "Guardar" de la pantalla de intervención?

En la versión actual, al pulsar el botón sale un mensaje (un Toast) que dice "Registro guardado en el servidor" y se cierra la pantalla. Es **una simulación visual**. La lógica para guardar realmente la intervención en la base de datos requeriría una entidad IntervencionEntity, su DAO y su repositorio, igual que tengo con Vehículos. Lo tenía pensado pero no me dio tiempo a implementarlo.

11. ¿Cómo gestiona que la base de datos no se ejecute en el hilo principal?

En el VehiculoRepository creo un **ExecutorService con Executors.newFixedThreadPool(4)**. Cuando llamo a insert, update o delete, en lugar de hacerlo directamente, le digo al executor que lo ejecute en segundo plano. Así la pantalla no se queda bloqueada esperando. Para las consultas de lectura uso LiveData, que ya gestiona los hilos por debajo de forma automática.

12. ¿Su app pide permisos de ubicación? ¿Cómo los gestiona?

En el AndroidManifest tengo declarados ACCESS_FINE_LOCATION y ACCESS_COARSE_LOCATION. **En la versión actual no estoy haciendo la solicitud en runtime**, que es lo que pide Android desde la versión 6. Como las coordenadas las genero de forma simulada, no me hacía falta. Cuando integre el GPS real tendré que añadir la solicitud de permisos en runtime, que sé que es obligatoria.

BLOQUE 3 · Preguntas de DECISIONES Y DISEÑO

13. ¿Por qué solo Android? ¿No se planteó hacerlo multiplataforma?

Por dos motivos. Primero, en mi retén **todos los dispositivos son Android**, así que para el caso real no me hacía falta iOS. Y segundo, hacer multiplataforma con Flutter o React Native habría supuesto aprender otro framework y duplicar el trabajo. Quería centrarme en hacer la app para Android lo mejor posible.

14. ¿Por qué SQLite y no una base de datos en la nube?

Porque el requisito principal era **que la app funcionara sin internet**. Las intervenciones forestales están en zonas con poca cobertura, y si la app necesita conexión para abrir o guardar datos, no me sirve. Una base de datos local me garantiza que la app funcione siempre. La sincronización con servidor la tengo planteada para la siguiente versión, pero **siempre como complemento, no como dependencia**.

15. ¿Ha pensado en la seguridad de los datos? ¿La base de datos está cifrada?

En esta versión no, la base de datos guarda los datos en texto plano dentro del almacenamiento privado de la app — al que solo puede acceder mi propia app. Para una versión real con datos reales del personal, sé que **habría que usar SQLCipher para cifrarla**, y añadir login con huella o PIN. Lo tengo identificado como mejora obligatoria antes de un despliegue real.

16. ¿Cumple el RGPD? Está guardando nombres de personas.

En la versión actual **los nombres son ficticios**, así que no hay datos personales reales. Para una versión real, sé que tendría que pedir consentimiento explícito al personal, cifrar la base de datos, tener una política de privacidad clara y permitir borrar los datos si alguien lo pide.

17. ¿Cómo se hace el login? ¿Hay seguridad?

En la versión actual el "login" es **simbólico**: hay una pantalla con campos de usuario y contraseña, pero al pulsar "Iniciar Turno" simplemente se pasa al menú principal, sin validar nada. Lo dejé así porque no había una base de usuarios real todavía. Para la versión con sistema de roles (v2.0 del roadmap) tendría que añadir una tabla de usuarios con contraseñas cifradas y validación en condiciones.

BLOQUE 4 · Preguntas sobre EL PROYECTO en general

18. ¿Qué es lo que más le ha costado del proyecto?

Sin duda, **conseguir que Hilt y Room funcionen juntos**. Por separado los iba pillando, pero hacerlos convivir — que la base de datos sea Singleton, que el DAO se inyecte donde toca, que el Repository se cree solo — me dio bastantes quebraderos de cabeza. Tuve un error al arrancar la app que tardé varias sesiones en localizar: era un fallo en el manifest donde no estaba declarada la clase RetenDigitalApp. Una vez resuelto, todo encajó.

19. ¿Si pudiera empezar de nuevo, qué haría diferente?

Tres cosas:

Primero, **empezaría por implementar Room para todas las entidades a la vez**, no módulo a módulo. Así no me habría quedado a medias.

Segundo, **haría el GPS real desde el principio**, no lo dejaría para el final.

Y tercero, **planificaría mejor el tiempo**. Subestimé lo que iba a costar configurar Hilt.

20. ¿Ha enseñado la app a sus compañeros del retén?

Sí, he hecho pruebas informales. Los comentarios que más me sirvieron fueron: que los botones tenían que ser más grandes para usarse con guantes (lo cambié), que el modo oscuro era importante por la batería y por verla mejor al sol, y que la pantalla de intervención debía priorizar la captura GPS por encima de cualquier otra cosa. Para una versión más completa, me gustaría hacer una **prueba real durante un turno entero**.

21. ¿Qué metodología ha seguido para desarrollar?

He trabajado con un enfoque **incremental**: ir haciendo módulos uno a uno, probarlos y luego pasar al siguiente. No he hecho Scrum formal con sprints porque era un proyecto individual y no tenía sentido. Pero sí he seguido la idea ágil: **iteraciones cortas, probar cada cosa antes de pasar a la siguiente, y usar Git para llevar el control de cambios**.

22. ¿Ha medido cuánto tiempo ahorraría su app frente al papel?

No tengo medidas formales todavía, lo reconozco. Por experiencia mía, una consulta de inventario que en papel puede llevar 5-10 minutos buscando entre carpetas, en la app sería instantánea. Y un parte oficial al final de turno, que ahora son 15-20 minutos a mano, podría reducirse mucho con exportación a PDF.

BLOQUE 5 · Preguntas DIFÍCILES o "trampa"

23. ¿Por qué este es un proyecto de DAM y no de DAW?

Porque es una **aplicación móvil nativa Android**, no una web. Usa el ciclo de vida de Android, las Activities, los Fragments, accede al GPS del dispositivo, guarda datos en la memoria del propio móvil... Una versión web no podría garantizar el funcionamiento sin internet ni acceder al GPS con la misma fluidez. La razón de DAM es exactamente este tipo de aplicación.

24. ¿No hay ya apps comerciales que hagan esto?

Hay sistemas de gestión de emergencias, sí, pero suelen ser **de gran formato y para mando central**, no para el equipo en campo. Y son productos comerciales caros que no se adaptan bien a las necesidades de un retén pequeño. Mi diferencial es que la app está **diseñada por alguien que trabaja en el retén**, prioriza el funcionamiento sin internet y se adapta a la operativa local concreta — UTM Huso 30T, terminología propia, etc.

25. La métrica de "12% → 3,5% de batería" que menciona, ¿cómo la ha medido?

Es una estimación basada en lecturas del medidor de batería de **Android Studio**, no una medición ultra-precisa con instrumental especializado. La hice comparando dos enfoques distintos del GPS: uno con actualizaciones constantes y otro bajo demanda. Para una métrica más rigurosa habría que usar herramientas de profiling avanzadas, lo reconozco.

26. ¿Ha subido su API Key de Google Maps a GitHub?

Buena pregunta y muy importante. En el manifest tengo un placeholder ("AIzaSyA_TU_LLAVE_AQUI") en lugar de una clave real. Soy consciente de que las API Keys nunca deben subirse a un repositorio público. Para una versión productiva habría que **usar variables de entorno o el archivo local.properties**, que está excluido de Git por defecto.

27. Veo que su app usa "fallbackToDestructiveMigration". ¿Sabe qué significa?

Sí. Significa que si en el futuro cambio la estructura de la base de datos (por ejemplo, añado una columna nueva), Room **borra la base de datos antigua y crea una nueva** en lugar de intentar migrar los datos. Lo puse así porque en desarrollo es cómodo y los datos son ficticios. **Para una versión productiva NO se debe usar**, porque borraría los datos reales del usuario. Habría que escribir migraciones explícitas.

28. ¿Y si me preguntan algo que no sé?

Plantilla mental para responder:

✓ "Esa parte no la profundicé en esta versión, pero la idea sería [explicación general razonada]. Es algo que tengo en la lista para la siguiente iteración."

✓ "Soy consciente de esa limitación. Como decía en el roadmap, está identificada como mejora futura."

✓ "Buena pregunta, déjenme pensar un momento..." (le da 5 segundos de oro).

✗ **NO diga:** "no sé", "ni idea", "no me acuerdo".

✗ **NO se invente nada.** Se nota muchísimo y queda peor que reconocer que no sabe.

BLOQUE 6 · Si no hay preguntas o el tiempo se acaba

Si dicen "no tenemos más preguntas":

"Muchas gracias por su tiempo y por la oportunidad de presentarles el proyecto." **NADA MÁS.** No se alargue. Cierre con elegancia y deje que el tribunal corte la sesión.

Si se queda en blanco:

"¿Les importaría reformular la pregunta para asegurarme de que la entiendo bien?" Esto le da 10-15 segundos para pensar y muestra interés, no debilidad.

 **Recordatorio final:** el tribunal sabe que es un alumno de DAM aprendiendo. **NO esperan que sepa todo.** Lo que valoran es:

1. Que entienda **lo que ha hecho.**
2. Que reconozca **lo que NO ha hecho** y por qué.
3. Que tenga **claras las mejoras futuras.**
4. Que **se exprese con honestidad y seguridad.**

Usted ha trabajado en este proyecto durante meses. **Sabe más del tema que nadie en esa sala.** Hable con tranquilidad. ¡Mucha suerte!  